ΜΥΥ 802 Μεταφραστές
Τελική Αναφορά Compiler Greek ++

Κιοσσές Αθανάσιος ΑΜ: 5254
Σιούλας Αλέξανδρος ΑΜ: 5349

Pipeline Μεταγλώττισης

Ο κώδικας οργανώνεται σε πέντε κύριες φάσεις:

- **Λεκτική Ανάλυση (Lexer):** Μετατρέπει τον πηγαίο κώδικα Greek++ σε μια ακολουθία από "λεκτικές μονάδες" (tokens).
- **Συντακτική Ανάλυση (Parser):** Παίρνει τις λεκτικές μονάδες και χτίζει ένα "Αφηρημένο Συντακτικό Δέντρο" (Abstract Syntax Tree - AST), επαληθεύοντας ότι ο κώδικας ακολουθεί τη γραμματική της Greek++.
- **Κατασκευή Πίνακα Συμβόλων (Static Pass - build_symbol_table):** Πριν την παραγωγή ενδιάμεσου κώδικα, γίνεται μια στατική ανάλυση του AST για να συλλεχθούν πληροφορίες για τις παραμέτρους των συναρτήσεων/διαδικασιών (στο param_map). Αυτό είναι κρίσιμο για τη σωστή διαχείριση των παραμέτρων κατά την παραγωγή κώδικα.
- **Παραγωγή Ενδιάμεσου Κώδικα (CodeGenerator):** Διασχίζει το AST και παράγει τετράδες (quadruples). Κατά τη διάρκεια αυτής της φάσης, ο SymbolTable ενημερώνεται δυναμικά με μεταβλητές, παραμέτρους, προσωρινές μεταβλητές, και πληροφορίες εμβέλειας (scopes).
- **Παραγωγή Τελικού Κώδικα (FinalCodeGenerator):** Μετατρέπει τις τετράδες σε κώδικα Assembly RISC-V, χρησιμοποιώντας τις πληροφορίες από τον SymbolTable (κυρίως τα closed_scopes και τα υπολογισμένα frame_lengths).

Στην main() εκτελούνται διαδοχικά αυτά τα βήματα, με ενδιάμεση αποθήκευση σε αρχεία .int, .sym και .asm.

Ανάλυση Κλάσεων και Συναρτήσεων

1. Lexer (Λεκτικός Αναλυτής)

- **Σκοπός**: Να διαβάσει τον πηγαίο κώδικα source_code χαρακτήρα προς χαρακτήρα και να τον χωρίσει σε λεκτικές μονάδες (tokens).
- **__init__(self, source_code)**:
    - Αρχικοποιεί την κατάσταση του lexer: τον πηγαίο κώδικα, την τρέχουσα θέση (position), τη γραμμή (line), και τον τρέχοντα χαρακτήρα (current_char).
    - Ορίζει δύο σημαντικά λεξικά:
        - reserved_words: Αντιστοιχίζει τις δεσμευμένες λέξεις της Greek++ (π.χ., 'πρόγραμμα') στους τύπους των tokens τους (π.χ., 'PROGRAM').
        - symbols: Αντιστοιχίζει σύμβολα και τελεστές (π.χ., '+', ':=') στους τύπους των tokens τους (π.χ., 'PLUS', 'ASSIGN').

- **advance(self)**: Προχωρά στον επόμενο χαρακτήρα του πηγαίου κώδικα. Ενημερώνει τον μετρητή γραμμής αν συναντήσει αλλαγή γραμμής (\n).
- **skip_spaces(self)**: Παρακάμπτει κενά, tabs, και αλλαγές γραμμής.
- **skip_comment(self)**: Παρακάμπτει σχόλια που βρίσκονται μεταξύ { και }.
- **get_identifier(self)**:
    - Διαβάζει μια ακολουθία από αλφαριθμητικούς χαρακτήρες (και ελληνικούς με τόνους, _) που ξεκινούν με γράμμα.
    - Ελέγχει αν η αναγνωρισμένη ακολουθία είναι δεσμευμένη λέξη (από το self.reserved_words).
    - Επιστρέφει ένα tuple: (TYPE, value), π.χ., ('PROGRAM', 'πρόγραμμα') ή ('ID', 'myVar').
- **get_integer(self)**: Διαβάζει μια ακολουθία ψηφίων και την επιστρέφει ως token τύπου 'INTEGER'.
- **get_symbol(self)**:
    - Διαβάζει τον τρέχοντα χαρακτήρα.
    - Ελέγχει αν, σε συνδυασμό με τον επόμενο, σχηματίζει ένα διπλό σύμβολο (π.χ., ':=', '<=').
    - Επιστρέφει το αντίστοιχο token από το self.symbols ή 'UNKNOWN' αν δεν αναγνωριστεί.
- **get_next_token(self)**:
    - Είναι η κύρια συνάρτηση που αποφασίζει ποια βοηθητική συνάρτηση (get_identifier, get_integer, get_symbol) θα καλέσει, βασιζόμενη στον τρέχοντα χαρακτήρα.
    - Παρακάμπτει κενά και σχόλια.
    - Επιστρέφει το επόμενο token ή 'EOF' (End Of File) token αν έχει φτάσει στο τέλος του κώδικα. Τα tokens είναι συνήθως tuples της μορφής (TYPE, value, line_number).
- **tokenize(self)**: Καλεί επαναληπτικά την get_next_token() μέχρι να λάβει 'EOF' και συγκεντρώνει όλα τα tokens σε μια λίστα.

2. Parser (Συντακτικός Αναλυτής)

- **Σκοπός**: Να πάρει τη λίστα των tokens από τον Lexer και να δημιουργήσει ένα Abstract Syntax Tree (AST) σύμφωνα με τη γραμματική της Greek++. Αν ο κώδικας δεν ακολουθεί τη γραμματική, προκαλεί ParserError.
- **__init__(self, tokens)**: Αρχικοποιεί τον parser με τη λίστα των tokens, την τρέχουσα θέση (pos) και το τρέχον token (current_token).
- **error(self, message)**: Δημιουργεί ένα μήνυμα λάθους που περιλαμβάνει πληροφορίες για το token που προκάλεσε το σφάλμα και τη γραμμή του.

- **parse_next_token(self, token_type)**: "Καταναλώνει" το current_token αν ο τύπος του ταιριάζει με τον token_type. Αν δεν ταιριάζει, καλεί την self.error(). Προχωρά στο επόμενο token.
- **parse(self)**: Η κύρια μέθοδος εκκίνησης της συντακτικής ανάλυσης. Καλεί την program().
- **Μέθοδοι program(), programblock(), declarations(), ... optional_sign()**:
    - **program**: Ορίζει τη συνολική δομή ενός προγράμματος: τη λέξη-κλειδί "πρόγραμμα", ένα αναγνωριστικό (όνομα προγράμματος), και το κυρίως μπλοκ του προγράμματος.
    - **programblock**: Περιγράφει το περιεχόμενο ενός προγράμματος: δηλώσεις μεταβλητών, ορισμούς υποπρογραμμάτων, την έναρξη του κύριου τμήματος κώδικα (αρχή_προγράμματος), την ακολουθία εντολών, και το τέλος του προγράμματος (τέλος_προγράμματος).
    - **declarations**: Μια (πιθανώς κενή) λίστα από δηλώσεις μεταβλητών, όπου κάθε δήλωση ξεκινά με "δήλωση" ακολουθούμενη από μια λίστα μεταβλητών.
    - **varlist**: Μια λίστα από ένα ή περισσότερα αναγνωριστικά (ονόματα μεταβλητών) χωρισμένα με κόμμα.
    - **subprograms**: Μια (πιθανώς κενή) λίστα από ορισμούς συναρτήσεων ή/και διαδικασιών.
    - **func**: Ορίζει μια συνάρτηση: τη λέξη-κλειδί "συνάρτηση", το όνομά της, τις τυπικές παραμέτρους σε παρενθέσεις, και το μπλοκ της συνάρτησης.
    - **proc**: Ορίζει μια διαδικασία: τη λέξη-κλειδί "διαδικασία", το όνομά της, τις τυπικές παραμέτρους σε παρενθέσεις, και το μπλοκ της διαδικασίας.
    - **formalparlist**: Η λίστα των τυπικών παραμέτρων μιας συνάρτησης/διαδικασίας. Μπορεί να είναι μια varlist ή κενή.
    - **funcblock**: Το σώμα μιας συνάρτησης. Περιλαμβάνει τη "διαπροσωπεία" (δηλώσεις εισόδου/εξόδου), τοπικές δηλώσεις, (εμφωλιασμένα) υποπρογράμματα, την αρχή_συνάρτησης, την ακολουθία εντολών, και το τέλος_συνάρτησης.
    - **procblock**: Το σώμα μιας διαδικασίας. Ανάλογο με το funcblock, αλλά με αρχή_διαδικασίας και τέλος_διαδικασίας.
    - **funcinput**: Η (προαιρετική) δήλωση παραμέτρων εισόδου ("είσοδος" ακολουθούμενη από varlist).
    - **funcoutput**: Η (προαιρετική) δήλωση παραμέτρων εξόδου ("έξοδος" ακολουθούμενη από varlist).
    - **sequence**: Μια ακολουθία από μία ή περισσότερες εντολές, χωρισμένες με ελληνικό ερωτηματικό (;).
    - **statement**: Μια μεμονωμένη εντολή. Μπορεί να είναι ανάθεση, if, while, repeat (do), for, read, write, ή call.
    - **assignment_stat**: Εντολή ανάθεσης: ID := expression.

- **if_stat**: Εντολή if: "εάν" συνθήκη "τότε" ακολουθία εντολών, (προαιρετικά) "αλλιώς" άλλη ακολουθία, και "εάν_τέλος".
- **elsepart**: Το προαιρετικό "αλλιώς" μέρος μιας εντολής if.
- **while_stat**: Βρόχος while: "όσο" συνθήκη "επανάλαβε" ακολουθία εντολών "όσο_τέλος".
- **do_stat**: Βρόχος repeat-until (εδώ "επανάλαβε-μέχρι"): "επανάλαβε" ακολουθία εντολών "μέχρι" συνθήκη.
- **for_stat**: Βρόχος for: "για" ID ":=" αρχική-έκφραση "έως" τελική-έκφραση (προαιρετικά) βήμα "επανάλαβε" ακολουθία εντολών "για_τέλος".
- **step**: Το προαιρετικό βήμα ("με_βήμα" έκφραση) σε έναν βρόχο for.
- **print_stat**: Εντολή εκτύπωσης: "γράψε" έκφραση.
- **input_stat**: Εντολή εισόδου: "διάβασε" ID.
- **call_stat**: Εντολή κλήσης υποπρογράμματος: "εκτέλεσε" ID (προαιρετικά) με πραγματικές παραμέτρους.
- **idtail**: Το προαιρετικό μέρος μιας κλήσης συνάρτησης ή χρήσης μεταβλητής που είναι οι πραγματικές παράμετροι σε παρενθέσεις.
- **actualpars**: Οι πραγματικές παράμετροι μιας κλήσης, μέσα σε παρενθέσεις.
- **actualparlist**: Η λίστα των πραγματικών παραμέτρων, χωρισμένες με κόμμα. Μπορεί να είναι κενή.
- **actualparitem**: Ένα μεμονωμένο πραγματικό όρισμα. Μπορεί να είναι μια έκφραση (πέρασμα κατ' αξία) ή % ID (πέρασμα κατ' αναφορά).
- **condition**: Μια λογική συνθήκη, που αποτελείται από έναν ή περισσότερους boolterm συνδεδεμένους με "ή".
- **boolterm**: Ένας όρος της συνθήκης, αποτελούμενος από έναν ή περισσότερους boolfactor συνδεδεμένους με "και".
- **boolfactor**: Ο βασικός παράγοντας μιας λογικής συνθήκης. Μπορεί να είναι: "όχι" [condition], [condition], ή μια σχεσιακή σύγκριση δύο εκφράσεων.
- **expression**: Μια αριθμητική έκφραση. Ξεκινά με ένα προαιρετικό πρόσημο, ακολουθούμενο από έναν term, και μετά (προαιρετικά) μια σειρά από term που συνδέονται με + ή -.
- **term**: Ένας όρος μιας αριθμητικής έκφρασης. Αποτελείται από έναν factor, ακολουθούμενο (προαιρετικά) από μια σειρά από factor που συνδέονται με * ή /.
- **factor**: Ο βασικός παράγοντας μιας αριθμητικής έκφρασης. Μπορεί να είναι ένας ακέραιος, μια έκφραση σε παρενθέσεις, ή ένα αναγνωριστικό (που μπορεί να είναι μεταβλητή ή κλήση συνάρτησης με idtail).
- **relational_oper**: Ένας σχεσιακός τελεστής (=, <=, >=, <>, <, >).
- **add_oper**: Προσθετικός τελεστής (+ ή -).

    - **mul_oper**: Πολλαπλασιαστικός τελεστής (* ή /).
    - **optional_sign**: Ένα προαιρετικό πρόσημο (+ ή -) στην αρχή μιας έκφρασης.
- **AST**: Το τελικό αποτέλεσμα του parser είναι μια δενδρική δομή (υλοποιημένη ως ένθετα tuples) που αναπαριστά τη συντακτική δομή του προγράμματος.

3. SymbolTable (Πίνακας Συμβόλων)

- **Σκοπός**: Να διαχειρίζεται πληροφορίες για αναγνωριστικά (μεταβλητές, παράμετροι, συναρτήσεις, διαδικασίες) και τις εμβέλειές τους (scopes). Είναι κρίσιμος για τη σημασιολογική ανάλυση και την παραγωγή κώδικα.
- **__init__(self)**:
    - scopes: Μια στοίβα (list) που κρατά τα ενεργά scopes. Κάθε scope είναι ένα λεξικό.
    - closed_scopes: Μια λίστα που αποθηκεύει τα scopes που έχουν κλείσει (μετά το exit_scope).
    - current_level: Το τρέχον επίπεδο ένθεσης των scopes.
- **enter_scope(self, name=None)**:
    - Αυξάνει το current_level.
    - Δημιουργεί ένα νέο λεξικό για το scope, αποθηκεύοντας το όνομά του (name, π.χ., όνομα συνάρτησης), το level, μια λίστα για τις entities (μεταβλητές, κλπ. σε αυτό το scope), το max_offset (για τον υπολογισμό του frame size), το start_quad (η τετράδα έναρξης του block), και το frame_length.
    - Προσθέτει το νέο scope στη στοίβα self.scopes.
- **exit_scope(self)**:
    - Αφαιρεί το τρέχον scope από τη στοίβα self.scopes.
    - Μειώνει το current_level.
    - Υπολογίζει το frame_length του scope που κλείνει. Αυτό είναι σημαντικό:
        - Για το κύριο πρόγραμμα, το frame length είναι το max_offset + 4 (ή τουλάχιστον 4).
        - Για συναρτήσεις/διαδικασίες, είναι το max_offset + 4 (ή τουλάχιστον 12+4 αν δεν υπάρχουν τοπικές/παράμετροι).
        - Το frame_length ευθυγραμμίζεται πάντα σε πολλαπλάσιο του 4.
    - Ενημερώνει την εγγραφή της συνάρτησης/διαδικασίας στο γονικό scope (αν υπάρχει) με το υπολογισμένο frame_length και start_quad.
    - Προσθέτει το scope που έκλεισε στη λίστα self.closed_scopes.
- **get_active_scopes_snapshot(self)**: Επιστρέφει ένα deepcopy της τρέχουσας στοίβας self.scopes. Χρησιμοποιείται από τον CodeGenerator για να αποθηκεύει την κατάσταση του πίνακα συμβόλων μετά από κάθε endblock τετράδα, για το αρχείο .sym.

- **_alloc_offset(self)**: Υπολογίζει και επιστρέφει το επόμενο διαθέσιμο offset μέσα στο τρέχον scope. Το πρώτο offset είναι 12 (για να αφήσει χώρο για return address, static link, κλπ.), και τα επόμενα αυξάνονται κατά 4 (μέγεθος ακεραίου/διεύθυνσης). Ενημερώνει το max_offset του τρέχοντος scope.
- **lookup(self, name)**:
    - Αναζητά ένα αναγνωριστικό (name) ξεκινώντας από το τρέχον scope και προχωρώντας προς τα έξω (προς τα γονικά scopes) μέχρι το global scope.
    - Αν βρεθεί, επιστρέφει την εγγραφή (λεξικό) της οντότητας.
    - Αν δεν βρεθεί, προκαλεί εξαίρεση (σημασιολογικό λάθος).
- **add_variable(self, name)**, **add_parameter(self, name, mode)**, **add_temp(self, name)**: Προσθέτουν μια νέα οντότητα (μεταβλητή, παράμετρος, προσωρινή μεταβλητή) στο τρέχον scope, μαζί με το είδος της, το όνομα, το υπολογισμένο offset, και (για παραμέτρους) τον τρόπο περάσματος (parMode).
- **find_closed_scope(self, name, expected_level)**: Αναζητά ένα συγκεκριμένο κλειστό scope με βάση το όνομα και το επίπεδο.
- **add_subprogram_entry(self, name, kind, arg_kinds_list, start_quad=None, frame_length=None)**: Προσθέτει μια εγγραφή για μια συνάρτηση ή διαδικασία στο *τρέχον ενεργό scope*. Αυτό σημαίνει ότι η *δήλωση* μιας εμφωλιασμένης συνάρτησης προστίθεται στο scope της περιβάλλουσας συνάρτησης.

4. build_symbol_table(node, symtab, param_map) (Συνάρτηση - Στατικό Πέρασμα)

- **Σκοπός**: Να διασχίσει το AST *πριν* την παραγωγή ενδιάμεσου κώδικα, με κύριο στόχο να γεμίσει το param_map.
- **param_map**: Ένα λεξικό που αντιστοιχίζει ονόματα συναρτήσεων/διαδικασιών σε μια λίστα πληροφοριών για τις παραμέτρους τους. Κάθε στοιχείο της λίστας είναι ένα tuple (param_name, mode, type), όπου mode είναι 'CV' (Call by Value) ή 'REF' (Call by Reference). Το type δεν χρησιμοποιείται αφού έχουμε μόνο ακεραίους.
- **Λειτουργία**:
    - Είναι μια αναδρομική συνάρτηση που δέχεται έναν κόμβο του AST, την *κενή* symtab (δεν τη γεμίζει ουσιαστικά εδώ, αυτό γίνεται στον CodeGenerator), και το *κενό* param_map.
    - Όταν συναντά έναν κόμβο 'function' ή 'procedure':
        - Παίρνει το όνομα της συνάρτησης/διαδικασίας.
        - Εξετάζει τους κόμβους func_input και func_output (που είναι varlist κόμβοι).
        - Για κάθε αναγνωριστικό στις παραμέτρους εισόδου, προσθέτει μια εγγραφή (param_name, 'CV', None) στο current_pm_entries.
        - Για κάθε αναγνωριστικό στις παραμέτρους εξόδου, προσθέτει μια εγγραφή (param_name, 'REF', None).
        - Αποθηκεύει το current_pm_entries στο param_map με κλειδί το όνομα της συνάρτησης/διαδικασίας.

    - Καλεί τον εαυτό της αναδρομικά για τα παιδιά του τρέχοντος κόμβου, και κυρίως για τα nested_subprograms_node, ώστε να καταγραφούν οι παράμετροι και των εμφωλιασμένων υποπρογραμμάτων.

- **Σημασία**: Το param_map είναι απαραίτητο στον CodeGenerator για να ξέρει τον τρόπο περάσματος κάθε παραμέτρου όταν προσθέτει παραμέτρους στον SymbolTable (κατά την είσοδο στο scope ενός υποπρογράμματος) και όταν παράγει par τετράδες για κλήσεις.

5. CodeGenerator (Παραγωγός Ενδιάμεσου Κώδικα)

- **Σκοπός**: Να διασχίσει το AST (που παρήγαγε ο Parser) και να παράγει ενδιάμεσο κώδικα με τη μορφή τετράδων (quadruples). Επίσης, αλληλεπιδρά με το SymbolTable για να προσθέτει μεταβλητές, να ελέγχει δηλώσεις, και να διαχειρίζεται scopes.
- **__init__(self, symtab, prog_name, param_map)**:
    - quads: Λίστα όπου θα αποθηκευτούν οι παραγόμενες τετράδες.
    - temp_count: Μετρητής για τη δημιουργία μοναδικών ονομάτων προσωρινών μεταβλητών (π.χ., t@1, t@2).
    - symtab: Το στιγμιότυπο του SymbolTable που αρχικοποιήθηκε πριν και ενδεχομένως χρησιμοποιήθηκε από το build_symbol_table (αν και το build_symbol_table κυρίως γεμίζει το param_map). Ο CodeGenerator είναι αυτός που κυρίως "γεμίζει" το SymbolTable με οντότητες και offsets.
    - program_name: Το όνομα του κύριου προγράμματος.
    - param_map: Το λεξικό που γέμισε η build_symbol_table, με πληροφορίες για τις παραμέτρους.
    - current_function: Κρατά το όνομα της τρέχουσας συνάρτησης/διαδικασίας που αναλύεται (χρήσιμο για τον χειρισμό της ανάθεσης στην ίδια τη συνάρτηση για επιστροφή τιμής).
    - symtab_snapshots: Λίστα για αποθήκευση "στιγμιότυπων" του symtab.scopes μετά από κάθε endblock, για το αρχείο .sym.
- **new_temp(self)**: Δημιουργεί ένα νέο όνομα προσωρινής μεταβλητής (π.χ., t@1), την προσθέτει στον SymbolTable μέσω της symtab.add_temp(), και επιστρέφει το όνομά της.
- **next_quad(self)**: Επιστρέφει τον αριθμό της επόμενης τετράδας που θα παραχθεί (δηλ. len(self.quads) + 1).
- **gen_quad(self, op, a1, a2, res)**:
    - Παράγει μια νέα τετράδα [quad_number, op, a1, a2, res] και την προσθέτει στη λίστα self.quads.
    - **Ειδικός χειρισμός για beginblock**: Ενημερώνει το start_quad στο SymbolTable για το τρέχον scope και για την εγγραφή της συνάρτησης/διαδικασίας στο γονικό scope.

    - **Ειδικός χειρισμός για endblock**: Παίρνει ένα "στιγμιότυπο" των ενεργών scopes από τον SymbolTable (symtab.get_active_scopes_snapshot()) και το αποθηκεύει στο self.symtab_snapshots.

- **backpatch(self, lst, target)**, **makelist(self, i)**, **merge(self, a, b)**: Βοηθητικές συναρτήσεις για τη διαχείριση λιστών από αριθμούς τετράδων που χρειάζονται "backpatching" (δηλ., η τέταρτη παράμετρός τους, που είναι ένας προορισμός άλματος, συμπληρώνεται αργότερα). Χρησιμοποιούνται κυρίως στις δομές ελέγχου (if, while).

- **generate_condition(self, node):**
    - Σκοπός: Να μετατρέψει έναν κόμβο λογικής συνθήκης (από το AST) σε τετράδες που, όταν εκτελεστούν, θα οδηγήσουν είτε σε ένα "true exit" είτε σε ένα "false exit". Επιστρέφει δύο λίστες από αριθμούς τετράδων:
    - truelist: Λίστα με τις τετράδες που αντιστοιχούν σε αληθή έκβαση της συνθήκης και χρειάζονται backpatching για να πηδήξουν στο "true" μονοπάτι εκτέλεσης.
    - falselist: Λίστα με τις τετράδες που αντιστοιχούν σε ψευδή έκβαση της συνθήκης και χρειάζονται backpatching για να πηδήξουν στο "false" μονοπάτι εκτέλεσης.
    - Ροή ανάλογα με τον τύπο του node[0]:
    - 'relation' (π.χ., expr1 < expr2):
    - Αξιολόγηση Εκφράσεων: Παράγει τετράδες για την expr1 (που αποθηκεύεται στο L) και την expr2 (που αποθηκεύεται στο R) χρησιμοποιώντας την self.generate_expression().
    - Τετράδα Σύγκρισης (Conditional Branch): Δημιουργεί μια τετράδα σύγκρισης με βάση τον τελεστή της σχέσης (π.χ., op_symbol, L, R, ''). Ο προορισμός ('') είναι κενός γιατί θα γίνει backpatch αργότερα. Ο αριθμός αυτής της τετράδας προστίθεται στην truelist.
    - Τετράδα Άλματος (Unconditional Jump για False): Δημιουργεί μια τετράδα jump, '', '', '' αμέσως μετά. Αυτή αντιπροσωπεύει την περίπτωση που η σύγκριση απέτυχε (false). Ο αριθμός αυτής της τετράδας προστίθεται στην falselist.
    - Επιστροφή: truelist, falselist.
    - 'and' (π.χ., cond1 and cond2):
    - Αξιολόγηση cond1: Καλεί αναδρομικά την self.generate_condition(cond1) για να πάρει t1 (truelist της cond1) και f1 (falselist της cond1).
    - Backpatch t1: Αν η cond1 είναι αληθής (t1), τότε πρέπει να αξιολογηθεί η cond2. Άρα, οι τετράδες της t1 γίνονται backpatch για να δείχνουν στην αρχή του κώδικα της cond2.
    - Αξιολόγηση cond2: Καλεί αναδρομικά την self.generate_condition(cond2) για να πάρει t2 και f2.
    - Επιστροφή:

    - truelist: Είναι η t2. Για να είναι ολόκληρη η and έκφραση αληθής, πρέπει και η cond1 και η cond2 να είναι αληθείς.
    - falselist: Είναι η merge(f1, f2). Η and έκφραση είναι ψευδής αν η cond1 είναι ψευδής (f1) *ή* αν η cond2 είναι ψευδής (f2).
    - 'or' (π.χ., cond1 or cond2):
    - Αξιολόγηση cond1: Καλεί αναδρομικά την self.generate_condition(cond1) για να πάρει t1 και f1.
    - Backpatch f1: Αν η cond1 είναι ψευδής (f1), τότε πρέπει να αξιολογηθεί η cond2. Άρα, οι τετράδες της f1 γίνονται backpatch για να δείχνουν στην αρχή του κώδικα της cond2.
    - Αξιολόγηση cond2: Καλεί αναδρομικά την self.generate_condition(cond2) για να πάρει t2 και f2.
    - Επιστροφή:
    - truelist: Είναι η merge(t1, t2). Η or έκφραση είναι αληθής αν η cond1 είναι αληθής (t1) *ή* αν η cond2 είναι αληθής (t2).
    - falselist: Είναι η f2. Για να είναι ολόκληρη η or έκφραση ψευδής, πρέπει και η cond1 και η cond2 να είναι ψευδείς.
    - 'not' (π.χ., not cond):
    - Αξιολόγηση cond: Καλεί αναδρομικά την self.generate_condition(cond) για να πάρει t (truelist της cond) και f (falselist της cond).
    - Επιστροφή (Αντιστροφή): Επιστρέφει f ως τη νέα truelist και t ως τη νέα falselist, αντιστρέφοντας ουσιαστικά τα μονοπάτια true/false.
- **generate_expression(self, node)**: Παράγει τετράδες για αριθμητικές εκφράσεις.
    - integer: Επιστρέφει την τιμή του ακεραίου ως string.
    - id:
        - Αναζητά το name στον SymbolTable.
        - Αν είναι κλήση συνάρτησης (tail is not None):
            - Ελέγχει αν είναι όντως συνάρτηση και αν ο αριθμός των ορισμάτων ταιριάζει.
            - Για κάθε όρισμα, παράγει κώδικα για την έκφραση (αν είναι by-value) ή παίρνει το όνομα (αν by-reference), και παράγει par τετράδα (par, value, mode, func_name).
            - Παράγει par για την τιμή επιστροφής (par, ret_temp, 'RET', func_name).
            - Παράγει call τετράδα.
            - Επιστρέφει το ret_temp.
        - Αν είναι απλή μεταβλητή: Επιστρέφει το όνομά της.

    - binop (π.χ., +, *): Παράγει αναδρομικά κώδικα για τα L και R, δημιουργεί ένα new_temp(), παράγει την τετράδα της πράξης (π.χ., +, L, R, t), και επιστρέφει το t.
    - unary (π.χ., -a): Αν είναι -, παράγει -, 0, val, t.
- **generate_statement(self, node)**: Παράγει τετράδες για εντολές.
    - assignment: Παράγει κώδικα για την έκφραση rhs. Αν το var_name είναι το όνομα της τρέχουσας συνάρτησης, παράγει retv, rhs, , (για επιστροφή τιμής). Αλλιώς, παράγει :=, rhs, , var_name.
    - print: Παράγει κώδικα για την έκφραση και μετά out, val, ,.
    - read: Παράγει in, , , name.
    - call (διαδικασίας): Παρόμοια με την κλήση συνάρτησης στην generate_expression, αλλά χωρίς τιμή επιστροφής. Παράγει par για κάθε όρισμα και μετά call, , , proc_name.
    - if, while, for, do: Καλεί τις αντίστοιχες generate_if, generate_while, κλπ.
- **generate_sequence(self, seq_node)**: Καλεί την generate_statement για κάθε εντολή στην ακολουθία.
- **_process_declarations_and_subprograms(self, declarations_node, subprograms_node)**:
    - Μια βοηθητική εσωτερική μέθοδος.
    - Για κάθε varlist στις declarations_node, καλεί symtab.add_variable() για κάθε ID.
    - Για κάθε ορισμό υποπρογράμματος (func ή proc) στις subprograms_node, καλεί symtab.add_subprogram_entry() για να προσθέσει την εγγραφή της *δήλωσης* του υποπρογράμματος στον πίνακα συμβόλων του *τρέχοντος* scope.
- **generate_procedure(self, node)** και **generate_function(self, node)**:
    - Παίρνουν το όνομα και το block του υποπρογράμματος.
    - Κάνουν symtab.enter_scope(name=proc_name/func_name).
    - Ενημερώνουν το self.current_function.
    - Για κάθε τυπική παράμετρο (από το self.param_map), καλούν symtab.add_parameter().
    - Καλούν την _process_declarations_and_subprograms για τις τοπικές δηλώσεις και τους ορισμούς *εμφωλιασμένων* υποπρογραμμάτων.
    - *Αναδρομικά καλούν* τις generate_function/generate_procedure για κάθε *εμφωλιασμένο* υποπρόγραμμα που ορίστηκε.
    - Επαναφέρουν το self.current_function στο όνομα του τρέχοντος υποπρογράμματος (σε περίπτωση που άλλαξε από αναδρομική κλήση).
    - Παράγουν beginblock, name, ,.

    - Καλούν generate_sequence() για το σώμα του υποπρογράμματος.
    - Παράγουν endblock, name, ,.
    - Καλούν symtab.exit_scope().
    - Επαναφέρουν το self.current_function στην προηγούμενη τιμή του.
- **generate_program(self, node)**:
    - Παρόμοια λογική με τις generate_function/procedure αλλά για το κύριο πρόγραμμα.
    - Εισέρχεται στο global scope.
    - Επεξεργάζεται global declarations και top-level subprogram definitions (καλώντας _process_declarations_and_subprograms).
    - Καλεί αναδρομικά τις generate_function/procedure για τα top-level υποπρογράμματα.
    - Παράγει beginblock για το κύριο πρόγραμμα.
    - Παράγει κώδικα για την κύρια ακολουθία εντολών.
    - Παράγει halt, , , και endblock για το κύριο πρόγραμμα.
    - Εξέρχεται από το global scope.
- **generate_if(self, node):**
    - Παραγωγή Κώδικα Συνθήκης: Καλεί την generate_condition για να παράγει τετράδες για τη συνθήκη. Αυτό επιστρέφει μια "true list" (tlist - τετράδες που εκτελούνται αν η συνθήκη είναι αληθής) και μια "false list" (flist).
    - Backpatch True List: Οι τετράδες της tlist γίνονται backpatch ώστε να δείχνουν στην αρχή του κώδικα του then block.
    - Παραγωγή Κώδικα then Block: Παράγονται οι τετράδες για τις εντολές του then.
    - Χειρισμός else Block (αν υπάρχει):
        - Δημιουργείται μια τετράδα jump στο τέλος του then block για να παρακάμψει το else.
        - Οι τετράδες της flist γίνονται backpatch ώστε να δείχνουν στην αρχή του else block.
        - Παράγεται ο κώδικας για το else block.
        - Η προηγούμενη jump τετράδα (που παρακάμπτει το else) γίνεται backpatch για να δείχνει μετά το τέλος του else block.
    - Χειρισμός Χωρίς else Block: Οι τετράδες της flist γίνονται backpatch ώστε να δείχνουν στην εντολή μετά το if statement (παρακάμπτοντας το then block).
- **generate_while(self, node):**
    - Σημείο Επανεκκίνησης Βρόχου: Καταγράφεται η ετικέτα (αριθμός τετράδας) για την αρχή της αξιολόγησης της συνθήκης.

    - Παραγωγή Κώδικα Συνθήκης: Καλείται η generate_condition. Επιστρέφει tlist (για είσοδο στο σώμα) και flist (για έξοδο από τον βρόχο).
    - Backpatch True List: Οι τετράδες της tlist γίνονται backpatch για να δείχνουν στην αρχή του σώματος του βρόχου.
    - Παραγωγή Κώδικα Σώματος Βρόχου: Παράγονται οι τετράδες για τις εντολές του βρόχου.
    - Άλμα στην Αρχή: Δημιουργείται μια τετράδα jump που πηδάει πίσω στο σημείο επανεκκίνησης (για επανεκτίμηση της συνθήκης).
    - Backpatch False List: Οι τετράδες της flist γίνονται backpatch για να δείχνουν στην εντολή μετά τον βρόχο while (έξοδος).
- **generate_for(self, node):**
    - Αρχικοποίηση Μετρητή: Παράγεται κώδικας για την αρχική τιμή του μετρητή του βρόχου και γίνεται ανάθεση.
    - Σημείο Ελέγχου Συνθήκης: Καταγράφεται η ετικέτα για τον έλεγχο της συνθήκης (μετρητής <= τελική_τιμή).
    - Παραγωγή Κώδικα Τελικής Τιμής: Παράγεται κώδικας για την έκφραση της τελικής τιμής.
    - Τετράδα Σύγκρισης και Άλματος: Παράγεται μια τετράδα που συγκρίνει τον μετρητή με την τελική τιμή.
        - Αν η συνθήκη ισχύει (π.χ., μετρητής <= τελική_τιμή), πηδάει στο σώμα του βρόχου.
        - Αμέσως μετά, παράγεται μια jump τετράδα που θα πηδήξει έξω από τον βρόχο αν η συνθήκη δεν ισχύει (αυτή η jump θα γίνει backpatch αργότερα).
    - Παραγωγή Κώδικα Σώματος Βρόχου: Παράγονται οι τετράδες για τις εντολές του βρόχου.
    - Ενημέρωση Μετρητή: Παράγεται κώδικας για την τιμή του βήματος (ή '1' αν δεν υπάρχει), προστίθεται στον μετρητή, και γίνεται ανάθεση πίσω στον μετρητή.
    - Άλμα στον Έλεγχο Συνθήκης: Δημιουργείται μια τετράδα jump που πηδάει πίσω στο σημείο ελέγχου της συνθήκης.
    - Backpatch Άλματος Εξόδου: Η jump τετράδα που δημιουργήθηκε στο βήμα 4 (για την περίπτωση που η συνθήκη αποτύχει αρχικά ή σε κάποια επανάληψη) γίνεται backpatch για να δείχνει στην εντολή μετά τον βρόχο for.
- **generate_do(self, node):**
    - Σημείο Αρχής Σώματος Βρόχου: Καταγράφεται η ετικέτα για την αρχή του σώματος (το σώμα εκτελείται πάντα τουλάχιστον μία φορά).
    - Παραγωγή Κώδικα Σώματος Βρόχου: Παράγονται οι τετράδες για τις εντολές του βρόχου.

    - Παραγωγή Κώδικα Συνθήκης Τερματισμού: Παράγεται κώδικας για την αξιολόγηση της συνθήκης μέχρι.
    - Τετράδα Σύγκρισης και Άλματος: Παράγεται μια τετράδα που αξιολογεί τη συνθήκη.
        - Αν η συνθήκη τερματισμού είναι αληθής, πηδάει στην εντολή *μετά* την επόμενη jump (δηλαδή, έξοδος από τον βρόχο).
        - Αν η συνθήκη τερματισμού είναι ψευδής, η εκτέλεση συνεχίζει στην επόμενη τετράδα.
    - Άλμα στην Αρχή Σώματος: Δημιουργείται μια τετράδα jump που πηδάει πίσω στην αρχή του σώματος του βρόχου (για επανάληψη). Η ετικέτα εξόδου (από το προηγούμενο βήμα) είναι αμέσως μετά από αυτό το jump.

6. FinalCodeGenerator (Παραγωγός Τελικού Κώδικα)

- **Σκοπός**: Να μετατρέψει τις τετράδες (που παρήγαγε ο CodeGenerator) σε κώδικα Assembly RISC-V.
- **__init__(self, quads, symtab, program_name)**:
    - quads: Η λίστα των τετράδων.
    - symtab: Το στιγμιότυπο του SymbolTable. Εδώ, κυρίως ενδιαφέρουν τα closed_scopes που περιέχουν τις τελικές πληροφορίες για offsets, frame lengths, κλπ.
    - program_name: Το όνομα του κύριου προγράμματος.
    - asm_code: Λίστα όπου θα αποθηκευτεί ο παραγόμενος κώδικας assembly.
    - frame_lengths: Ένα λεξικό που θα αντιστοιχίζει ονόματα scopes (program/function/procedure) στα υπολογισμένα μεγέθη των stack frames τους.
    - active_scope_level_for_final_code: Κρατά το επίπεδο του τρέχοντος scope κατά την παραγωγή assembly (ενημερώνεται σε κάθε beginblock).
    - Καλεί την self.compute_frame_lengths().
- **compute_frame_lengths(self)**:
    - Διασχίζει *όλα* τα self.symtab.closed_scopes.
    - Για κάθε scope, παίρνει το name και το max_offset.
    - Υπολογίζει το frame_len (παρόμοια λογική με το SymbolTable.exit_scope()) και το αποθηκεύει στο self.frame_lengths[name].
    - Επίσης, ενημερώνει το frame_length στην εγγραφή της συνάρτησης/διαδικασίας μέσα στις entities του global scope.
- **find_symbol_static_info(self, name)**:
    - Αναζητά πληροφορίες για ένα σύμβολο (name) στα symtab.closed_scopes (πρωτίστως) ή

στα symtab.scopes (δευτερευόντως, για σενάρια όπου ένα scope μπορεί να μην έχει κλείσει πλήρως κατά κάποιο snapshot).
    - Επιστρέφει ένα λεξικό με kind, level, offset, parMode.
- **get_operand_info(self, operand)**: Επιστρέφει πληροφορίες για έναν τελεστή (αν είναι literal ή symbol).
- **gnlvcode(self, var_info)**: (Generate Non-Local Variable access Code)
    - Παράγει κώδικα assembly για την πρόσβαση σε μια μη-τοπική μεταβλητή.
    - target_level: Το επίπεδο της μεταβλητής.
    - self.active_scope_level_for_final_code: Το επίπεδο της ρουτίνας που εκτελείται τώρα.
    - Η διαφορά level_diff καθορίζει πόσες φορές πρέπει να ακολουθήσει τον static link (lw t0, -4(sp) ή lw t0, -4(t0)) για να φτάσει στο σωστό stack frame.
    - Τέλος, προσθέτει το offset της μεταβλητής: addi t0, t0, -{var_info['offset']}. Το t0 τώρα περιέχει τη *διεύθυνση* της μεταβλητής.
- **loadvr(self, operand, target_reg, current_sp_or_fp_base='sp')**: (Load Value into Register)
    - Παράγει κώδικα για να φορτώσει την *τιμή* του operand στον target_reg.
    - literal: li target_reg, value.
    - variable/temp/parameter:
        - var_level == 0 (global): Πρόσβαση μέσω gp. lw target_reg, -offset(gp).
        - var_level == self.active_scope_level_for_final_code (local): Πρόσβαση μέσω sp. lw target_reg, -offset(sp).
        - else (non-local): Καλεί gnlvcode() για να πάρει τη διεύθυνση στο t0, μετά lw target_reg, 0(t0).
        - Αν είναι parMode == 'REF', κάνει ένα επιπλέον dereference (lw t0, 0(t0) και μετά lw target_reg, 0(t0) αν ήταν ήδη σε t0, ή lw target_reg, 0(address_reg)).
- **storerv(self, source_reg, target_variable_name, current_sp_or_fp_base='sp')**: (Store Register to Variable)
    - Παράγει κώδικα για να αποθηκεύσει την τιμή από τον source_reg στη μεταβλητή target_variable_name.
    - Η λογική για την εύρεση της διεύθυνσης της target_variable_name είναι παρόμοια με την loadvr (global, local, non-local, by-reference).
    - Τελική εντολή: sw source_reg, 0(address_reg) ή sw source_reg, -offset(base_reg).
- **_generate_newline_code(self)**: Βοηθητική για την εκτύπωση newline μετά από out.
- **generate_code(self)**: Η κύρια συνάρτηση παραγωγής τελικού κώδικα.

- Προετοιμασία: .data section (αν χρειάζεται για str_nl), .text, αρχικό άλμα j L{main_begin_quad_num}.
- Διασχίζει τις self.quads:
  - Για κάθε τετράδα, παράγει μια ετικέτα L{n_quad}:.
  - **beginblock**:
    - Ενημερώνει το self.active_scope_level_for_final_code.
    - Παίρνει το frame_len από το self.frame_lengths.
    - Για το program_name (main): addi sp, sp, frame_len και mv gp, sp.
    - Για συναρτήσεις/διαδικασίες: sw ra, 0(sp) (αποθήκευση return address).
  - **endblock**:
    - Για συναρτήσεις/διαδικασίες: lw ra, 0(sp) και jr ra.
  - **halt**: Syscall για τερματισμό.
  - **+, -, *, /**: loadvr για a1 (σε t1), loadvr για a2 (σε t2), εκτέλεση της RISC-V πράξης, storerv του αποτελέσματος (από t1) στο res.
  - **:=**: loadvr του a1 (σε t1), storerv του t1 στο res.
  - **<, <=, >, >=, =, <>**: loadvr για a1 (σε t1), loadvr για a2 (σε t2), αντίστοιχη RISC-V branch εντολή (π.χ., blt t1, t2, L{res}).
  - **jump**: j L{res}.
  - **in**: Syscall για διάβασμα ακεραίου, storerv του a0 (όπου επιστρέφεται η τιμή) στο res.
  - **out**: loadvr του a1 (σε a0), syscall για εκτύπωση ακεραίου, κώδικας για εκτύπωση newline.
  - **retv**: loadvr του a1 (σε t1). Φορτώνει τη διεύθυνση του χώρου επιστροφής από το stack (lw t0, -8(sp) - το sp εδώ είναι του καλούντος, που είναι το fp του καλούμενου). Αποθηκεύει το t1 σε αυτή τη διεύθυνση (sw t1, 0(t0)).
  - **par**:
    - callee = res.
    - temp_fp_register_to_use = "fp".
    - Αν είναι η πρώτη par για αυτόν τον callee:
      - addi fp, sp, callee_frame_len (Υπολογίζει πού θα είναι το sp του καλούμενου, και το βάζει στο fp -- αυτό το fp είναι προσωρινό για την προετοιμασία των παραμέτρων).

            - Αποθηκεύει τις παραμέτρους στο stack frame του *καλούμενου* (προσπελάσιμο μέσω του temp_fp_register_to_use):
                - RET: addi t0, sp, -temp_var_offset (διεύθυνση της προσωρινής μεταβλητής στο stack του *καλούντος*). sw t0, -8(fp) (αποθήκευση της *διεύθυνσης* στο slot επιστροφής).
                - in: loadvr του a1 σε t0. sw t0, -param_slot_offset(fp).
                - inout: Υπολογίζει τη *διεύθυνση* του a1 (με gnlvcode αν χρειάζεται, ή addi από sp/gp) και την βάζει στο t0. sw t0, -param_slot_offset(fp).
        - **call**:
            - callee_name = res. Παίρνει την ετικέτα του beginblock του callee.
            - sw sp, -4(fp) (Αποθηκεύει τον static link -- το sp του καλούντος -- στο stack frame του καλούμενου, προσπελάσιμο μέσω του fp που υπολογίστηκε στα par).
            - addi sp, sp, callee_frame_len (Προσαρμόζει το sp για να δημιουργήσει το νέο stack frame).
            - jal callee_begin_label.
            - addi sp, sp, -callee_frame_len (Επαναφέρει το sp μετά την επιστροφή).
    - Συνθέτει όλες τις γραμμές assembly.

7. Κύρια Λογική Εκτέλεσης (if __name__ == '__main__':)

- Διαχειρίζεται τα ορίσματα της γραμμής εντολών (όνομα αρχείου εισόδου).
- Ορίζει τα ονόματα των αρχείων εξόδου (.int, .sym, .asm).
- Διαβάζει τον πηγαίο κώδικα.
- **Εκτελεί την αλυσίδα μεταγλώττισης**:
    1. Lexer(src).tokenize() -> tokens
    2. Parser(tokens).parse() -> tree (AST)
    3. Εξάγει το program_name.
    4. Αρχικοποιεί symtab = SymbolTable() και param_map = {}.
    5. Καλεί build_symbol_table(tree, symtab, param_map) για να γεμίσει το param_map.
    6. Αρχικοποιεί codegen = CodeGenerator(symtab, program_name, param_map).

7. Καλεί codegen.generate_program(tree) για να παράγει τις quads και να γεμίσει πλήρως τον symtab (με scopes, entities, offsets, start_quads, & τα symtab_snapshots).
8. Γράφει τις τετράδες στο αρχείο .int.
9. Αρχικοποιεί final_gen = FinalCodeGenerator(quads, codegen.symtab, program_name). Ο constructor του FinalCodeGenerator καλεί την compute_frame_lengths(), η οποία διασχίζει τα codegen.symtab.closed_scopes και ενημερώνει/υπολογίζει τα frame_length για όλα τα scopes, αποθηκεύοντάς τα στο final_gen.frame_lengths.
10. Γράφει τις πληροφορίες του πίνακα συμβόλων (από τα codegen.symtab_snapshots και χρησιμοποιώντας τα final_gen.frame_lengths για τα μεγέθη των frames) στο αρχείο .sym.
11. Καλεί final_gen.generate_code() για να παράγει τις asm_lines.
12. Γράφει τον κώδικα assembly στο αρχείο .asm.

- Περιλαμβάνει χειρισμό εξαιρέσεων για σφάλματα κατά τη μεταγλώττιση.